Web Technologies

.....AND SECURITY

Stones Dalitso Chindipha

Chapters 9, 12

PHP Advanced: Superglobals, Form Handling

Objectives

1. $_GET and $_POST Superglobals
2. $_SERVER Array
3. PHP Form Handling
4. PHP File Handling
5. PHP OOP - Classes and Objects
6. PHP MySQL Database

PHP Global Variables - Superglobals

- **superglobal variables**: predefined associative arrays superglobal variables that allow the programmer to easily access HTTP headers, query string parameters, and other commonly needed information.
  - Super global variables are built-in variables that are always available in all scopes.
- The PHP superglobal variables are:
  - **$GLOBALS, $_SERVER**
  - $_REQUEST, $_FILES
  - **$_POST, $_GET**
  - $_ENV
  - $_COOKIE
  - $_SESSION

Name	Description
$GLOBALS	Array for storing data that needs superglobal scope
$_COOKIES	Array of cookie data passed to page via HTTP request
$_ENV	Array of server environment data
$_FILES	Array of file items uploaded to the server
$_GET	Array of query string data passed to the server via the URL
$_POST	Array of query string data passed to the server via the HTTP header
$_REQUEST	Array containing the contents of $_GET, $_POST, and $_COOKIES
$_SESSION	Array that contains session data
$_SERVER	Array containing information about the request and the server

TABLE 9.1 Sugerglobal Variables

Reading: Pgs 419 – 440 in main text book

PHP $GLOBALS

- **$GLOBALS** is a PHP super global variable which is used to access global variables from anywhere in the PHP script
  - This includes within functions or methods
- PHP stores all global variables in an array called $GLOBALS[index].
- The index holds the name of the variable.

```html
<!DOCTYPE html>
<html>
<body>

<?php
$x = 75;
$y = 25;

function addition() {
  $GLOBALS['z'] = $GLOBALS['x'] + $GLOBALS['y'];
}

addition();
echo $z;
?>

</body>
</html>
```

PHP $_SERVER

- **$_SERVER** is a PHP super global variable which holds information about headers, paths, and script locations
- A list of the most important elements that can go inside $_SERVER:
  - https://www.w3schools.com/php/php_superglobals_server.asp

```php
<?php
echo $_SERVER['PHP_SELF'];
echo "<br>";
echo $_SERVER['SERVER_NAME'];
echo "<br>";
echo $_SERVER['HTTP_HOST'];
echo "<br>";
echo $_SERVER['HTTP_REFERER'];
echo "<br>";
echo $_SERVER['HTTP_USER_AGENT'];
echo "<br>";
echo $_SERVER['SCRIPT_NAME'];
?>
```

```php
<!DOCTYPE html>
<html>
<body>

<?php
echo $_SERVER['PHP_SELF'];
echo "<br>";
echo $_SERVER['SERVER_NAME'];
echo "<br>";
echo $_SERVER['HTTP_HOST'];
echo "<br>";
echo $_SERVER['HTTP_REFERER'];
echo "<br>";
echo $_SERVER['HTTP_USER_AGENT'];
echo "<br>";
echo $_SERVER['SCRIPT_NAME'];
?>

</body>
</html>
```

/demo/demo_global_server.php
35.194.26.41
35.194.26.41
https://tryphp.w3schools.com/showphp.php?filename=demo_global_server
Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/101.0.4951.54 Safari/537.36
/demo/demo_global_server.php

PHP $_REQUEST

- **PHP $_REQUEST** is a PHP super global variable which is used to collect data after submitting an HTML form.
  -
```php
<?php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
  // collect value of input field
  $name = $_REQUEST['fname'];
  if (empty($name)) {
    echo "Name is empty";
  } else {
    echo $name;
  }
}
?>
```

```html
<!DOCTYPE html>
<html>
<body>

<form method="post" action="<?php echo $_SERVER['PHP_SELF'];?>">
  Name: <input type="text" name="fname">
  <input type="submit">
</form>

<?php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    // collect value of input field
    $name = htmlspecialchars($_REQUEST['fname']);
    if (empty($name)) {
        echo "Name is empty";
    } else {
        echo $name;
    }
}
?>

</body>
</html>
```

Name: Submit

Name is empty

$_GET and $_POST

- The **$_GET** and **$_POST** arrays are the most important superglobal variables in PHP since they allow the programmer to access data sent by the client in a query string.
- **PHP $_POST** is a PHP super global variable which is used to collect form data after submitting an HTML form with method="post".
  - $_POST is also widely used to pass variables.
  - Post requests are parsed into the $POST array
- **PHP $_GET** is a PHP super global variable which is used to collect form data after submitting an HTML form with method="get".
  - **$_GET** can also collect data sent in the URL.
  - Get requests parse query strings into the $_GET array

```html
<!DOCTYPE html>
<html>
<body>

<form method="post" action="<?php echo $_SERVER['PHP_SELF'];?>">
  Name: <input type="text" name="fname">
  <input type="submit">
</form>

<?php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    // collect value of input field
    $name = $_POST['fname'];
    if (empty($name)) {
        echo "Name is empty";
    } else {
        echo $name;
    }
}
?>

</body>
</html>
```

Name: Submit

John Doe

```
<html>
<body>

<?php
echo "Study " . $_GET['subject'] . " at " . $_GET['web'];
?>

</body>
</html>
```

```
<!DOCTYPE html>
<html>
<body>

<a href="test_get.php?subject=PHP&web=W3schools.com">Test $GET</a>

</body>
</html>
```

Test $GET

Study PHP at W3schools.com

HTML (client)

```
<form action="processLogin.php" method="GET">
    Name <input type="text" name="uname" />
    Pass <input type="text" name="pass" />
    <input type="submit">
</form>
```

Browser (client)

Name ricardo Pass pw01 Submit Query

HTTP request

```
GET processLogin.php?uname=ricardo&pass=pw01
```

PHP (server)

```
// within fileprocessLogin.php
echo $_GET["uname"]; // outputs ricardo
echo $_GET["pass"];  // outputs pw01
```

FIGURE 9.5 Illustration of flow from HTML, to request, to PHP's $_GET array

PHP Form Handling: Q-Recap

- The PHP superglobals $_GET and $_POST are used to collect form-data.
- GET vs. POST
  - Both GET and POST create an array (e.g. array(key1 => value1, key2 => value2, key3 => value3, ...)).
    - This array holds key/value pairs, where keys are the names of the form controls and values are the input data from the user.
  - Both GET and POST are treated as $_GET and $_POST.
    - These are superglobals, which means that they are always accessible, regardless of scope - and you can access them from any function, class or file without having to do anything special.
  - $_GET is an array of variables passed to the current script via the URL parameters.
  - $_POST is an array of variables passed to the current script via the HTTP POST method.

PHP - A Simple HTML Form

```html
<!DOCTYPE HTML>
<html>
<body>

<form action="welcome.php" method="post">
Name: <input type="text" name="name"><br>
E-mail: <input type="text" name="email"><br>
<input type="submit">
</form>

</body>
</html>
```

Name:
E-mail:
Submit

```php
<html>
<body>

Welcome <?php echo $_POST["name"]; ?><br>
Your email address is: <?php echo $_POST["email"]; ?>

</body>
</html>
```

```
!DOCTYPE HTML>
html>
body>

form action="welcome_get.php" method="get">
ame: <input type="text" name="name"><br>
-mail: <input type="text" name="email"><br>
input type="submit">
/form>

/body>
/html>
```

Welcome Jane Doe
Your email address is: Jane@bitbucket.com

```
<html>
<body>

Welcome <?php echo $_GET["name"]; ?><br>
Your email address is: <?php echo $_GET["email"]; ?>

</body>
</html>
```

When to use GET?

- Information sent from a form with the GET method is **visible to everyone** (all variable names and values are displayed in the URL).
- GET also has limits on the amount of information to send.
  - The limitation is about 2000 characters
- GET may be used for sending non-sensitive data.
- Because the variables are displayed in the URL, it is possible to bookmark the page.
- **Note: GET should NEVER be used for sending passwords or other sensitive information!**

When to use POST?

- Information sent from a form with the POST method is **invisible to others**
  - all names/values are embedded within the body of the HTTP request
- POST has no limits on the amount of information to send.
- POST supports advanced functionality such as support for multi-part binary input while uploading files to server.
- Because the variables are not displayed in the URL, it is not possible to bookmark the page.
- **Developers prefer POST for sending form data.**

PHP Form Validation

- Proper validation of form data is important to protect your form from hackers and spammers!
- Some validation rules to consider on a form:

Field	Validation Rules
Name	Required. + Must only contain letters and whitespace
E-mail	Required. + Must contain a valid email address (with @ and .)
Website	Optional. If present, it must contain a valid URL
Comment	Optional. Multi-line input field (textarea)
Gender	Required. Must select one

PHP Form Validation Example

* required field

Name: *

E-mail: *

Website:

Comment:

Gender: Female Male Other *

Submit

Your Input:

Form Code

- **Text Fields:**
  - ```
    Name: <input type="text" name="name">
    E-mail: <input type="text" name="email">
    Website: <input type="text" name="website">
    Comment: <textarea name="comment" rows="5" cols="40"></textarea>
    ```
- **Radio Buttons:**
  - ```
    Gender:
    <input type="radio" name="gender" value="female">Female
    <input type="radio" name="gender" value="male">Male
    <input type="radio" name="gender" value="other">Other
    ```
- **The Form Element:**
  - ```
    <form method="post" action="<?php echo htmlspecialchars($_SERVER["PHP_SELF"]);?>">
    ```
    ```

The Form Element

- The Form Element:
  - `<form method="post" action="<?php echo htmlspecialchars($_SERVER["PHP_SELF"]);?>">`
- **$_SERVER["PHP_SELF"]:** returns the filename of the currently executing script.
  - the $_SERVER["PHP_SELF"] sends the submitted form data to the page itself.
- **htmlspecialchars()** function converts special characters to HTML entities.
  - This means that it will replace HTML characters like < and > with < and >.
    - This prevents attackers from exploiting the code by injecting HTML or Javascript code (Cross-site Scripting attacks) in forms.

Big Note on PHP Form Security

- The **$_SERVER["PHP_SELF"]** variable can be used by hackers!
- If PHP_SELF is used in your page then a user can enter a slash (/) and then some Cross Site Scripting (XSS) commands to execute.
  - **Cross-site scripting (XSS)** is a type of computer security vulnerability typically found in Web applications.
    - XSS enables attackers to inject client-side script into Web pages viewed by other users.

Study Case: $_SERVER["PHP_SELF"] Exploit

- Assume we have the following form in a page named "test_form.php":
  - `<form method="post" action="<?php echo $_SERVER["PHP_SELF"];?>">`
- if a user enters the normal URL in the address bar like "http://www.example.com/test_form.php", the above code will be translated to:
  - `<form method="post" action="test_form.php">`
- So far, so good. However, consider that a user enters the following URL in the address bar:
  - http://www.example.com/test_form.php/%22%3E%3Cscript%3Ealert('hacked')%3C/script%3E
- In this case, the above code will be translated to:
  - `<form method="post" action="test_form.php/"><script>alert('hacked')</script>`

How To Avoid $_SERVER["PHP_SELF"] Exploits?

- $_SERVER["PHP_SELF"] exploits can be avoided by using the **htmlspecialchars()** function.
- The form code should look like this:
  - `<form method="post" action="<?php echo htmlspecialchars($_SERVER["PHP_SELF"]);?>">`
    - **Recall: htmlspecialchars() function converts special characters to HTML entities.**
- Now if the user tries to exploit the PHP_SELF variable, it will result in the following output:
  - `<form method="post" action="test_form.php/"><script>alert('hacked')</script>">`
    - **The exploit attempt fails, and no harm is done!**
    - The code is now safe to be displayed on a page or inside an e-mail.

Validate Form Data With PHP

- When validating a form, the first thing to do is to pass all variables through PHP's **htmlspecialchars()** function.
- In addition to this: when the user submits the form:
  - Strip unnecessary characters (extra space, tab, newline) from the user input data
    - Done with **PHP trim()** function
  - Remove backslashes (\) from the user input data
    - Done with the **PHP stripslashes()** function
- Furthermore, create a custom function that will do all the checking on your form

```php
<?php
// define variables and set to empty values
$name = $email = $gender = $comment = $website = "";

if ($_SERVER["REQUEST_METHOD"] == "POST") {
  $name = test_input($_POST["name"]);
  $email = test_input($_POST["email"]);
  $website = test_input($_POST["website"]);
  $comment = test_input($_POST["comment"]);
  $gender = test_input($_POST["gender"]);
}

function test_input($data) {
  $data = trim($data);
  $data = stripslashes($data);
  $data = htmlspecialchars($data);
  return $data;
}
?>
```

PHP Forms - Required Fields

- The next step is to make input fields required and create error messages if needed.
- From our example's validation rules, these fields cannot be empty and must be filled out in the HTML form.

Field	Validation Rules
Name	Required. + Must only contain letters and whitespace
E-mail	Required. + Must contain a valid email address (with @ and .)
Website	Optional. If present, it must contain a valid URL
Comment	Optional. Multi-line input field (textarea)
Gender	Required. Must select one

```php
<?php
// define variables and set to empty
values
$nameErr = $emailErr = $genderErr =
$websiteErr = "";
$name = $email = $gender = $comment =
$website = "";

if ($_SERVER["REQUEST_METHOD"] == "POST")
{
  if (empty($_POST["name"])) {
    $nameErr = "Name is required";
  } else {
    $name = test_input($_POST["name"]);
  }

  if (empty($_POST["email"])) {
    $emailErr = "Email is required";
  } else {
    $email = test_input($_POST["email"]);
  }
```

```
if (empty($_POST["website"])) {
    $website = "";
  } else {
    $website =
test_input($_POST["website"]);
  }

  if (empty($_POST["comment"])) {
    $comment = "";
  } else {
    $comment =
test_input($_POST["comment"]);
  }

  if (empty($_POST["gender"])) {
    $genderErr = "Gender is required";
  } else {
    $gender =
test_input($_POST["gender"]);
  }
}
?>
```

PHP - Display The Error Messages

- In the HTML form, add a little script after each required field, which generates the correct error message if needed:
  - i.e. if the user tries to submit the form without filling out the required fields
- The next step is to validate the input data by asking the right questions:
  - Does the Name field contain only letters and whitespace?
  - Does the E-mail field contain a valid e-mail address syntax?
  - If filled out, "Does the Website field contain a valid URL?".

DEMO

```
<form method="post" action="<?php echo htmlspecialchars($_SERVER["PHP_SELF"])
;?>">

Name: <input type="text" name="name">
<span class="error">* <?php echo $nameErr;?></span>
<br><br>
E-mail:
<input type="text" name="email">
<span class="error">* <?php echo $emailErr;?></span>
<br><br>
Website:
<input type="text" name="website">
<span class="error"><?php echo $websiteErr;?></span>
<br><br>
Comment: <textarea name="comment" rows="5" cols="40"></textarea>
<br><br>
Gender:
<input type="radio" name="gender" value="female">Female
<input type="radio" name="gender" value="male">Male
<input type="radio" name="gender" value="other">Other
<span class="error">* <?php echo $genderErr;?></span>
<br><br>
<input type="submit" name="submit" value="Submit">

</form>
```

PHP Forms - Validate E-mail and URL

- **PHP - Validate Name:** The code below shows a simple way to check if the name field only contains letters, dashes, apostrophes and whitespaces.
  - If the value of the name field is not valid, then store an error message:
  - ```
    $name = test_input($_POST["name"]);
    if (!preg_match("/^[a-zA-Z-' ]*$/",$name)) {
      $nameErr = "Only letters and white space allowed";
    }
    ```
    - **The preg_match() function searches a string for pattern, returning true if the pattern exists, and false otherwise.**
    ```

PHP - Validate E-mail

- The easiest and safest way to check whether an email address is well-formed is to use PHP's **filter_var()** function.
- In the code below, if the e-mail address is not well-formed, then store an error message:
  - ```
    $email = test_input($_POST["email"]);
    if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
      $emailErr = "Invalid email format";
    }
    ```

https://www.w3schools.com/php/func_filter_var.asp
    ```

PHP - Validate URL

- The code below shows a way to check if a URL address syntax is valid
  - The regular expression below also allows dashes in the URL
- If the URL address syntax is not valid, then store an error message:
  - ```
    $website = test_input($_POST["website"]);
    if (!preg_match("/\b(?:(?:https?|ftp):\/\/|www\.)[-a-z0-9+&@#\/%?=~_|!:,.;]*[-a-z0-9+&@#\/%=~_|]/i",$website)) {
      $websiteErr = "Invalid URL";
    }
    ```
    ```

```php
<?php
// define variables and set to empty values
$nameErr = $emailErr = $genderErr = $websiteErr = "";
$name = $email = $gender = $comment = $website = "";

if ($_SERVER["REQUEST_METHOD"] == "POST") {
  if (empty($_POST["name"])) {
    $nameErr = "Name is required";
  } else {
    $name = test_input($_POST["name"]);
    // check if name only contains letters and whitespace
    if (!preg_match("/^[a-zA-Z-' ]*$/",$name)) {
      $nameErr = "Only letters and white space allowed";
    }
  }

if (empty($_POST["email"])) {
    $emailErr = "Email is required";
  } else {
    $email = test_input($_POST["email"]);
    // check if e-mail address is well-formed
    if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
      $emailErr = "Invalid email format";
    }
  }
```

```
 if (empty($_POST["website"])) {
    $website = "";
  } else {
    $website = test_input($_POST["website"]);
    // check if URL address syntax is valid (this regular expression also allows dashes in the URL)
    if (!preg_match("/\b(?:(?:https?|ftp):\/\/|www\.)[-a-z0-9+&@#\/%?=~_|!:,.;]*[-a-z0-
9+&@#\/%=~_|]/i",$website)) {
      $websiteErr = "Invalid URL";
    }
  }

  if (empty($_POST["comment"])) {
    $comment = "";
  } else {
    $comment = test_input($_POST["comment"]);
  }

  if (empty($_POST["gender"])) {
    $genderErr = "Gender is required";
  } else {
    $gender = test_input($_POST["gender"]);
  }
}
?>
```

Reading Task: PHP Complete Form

- **The next step is to show how to prevent the form from emptying all the input fields when the user submits the form.**

https://www.w3schools.com/php/php_form_complete.asp